期货账户分析网站需求设计

- 1 需求分析
- 2 整体方案
  - 2.1 实现目标
  - 2.2 总体流程
  - 2.3 产品架构与模块设计
  - 2.4 界面效果
- 3 开发设计
  - 3.1 概要设计
  - 3.2 详细设计
  - 3.3 开发周期
- 4 非功能性需求
- 后续跟踪

1 需求分析

1.1 业务背景

在金融衍生品市场中，期货交易者需要实时监控和分析他们的账户状态和交易表现。为了提高交易效率和风险管理，开发一个期货账户分析网站是必要的。

1.2 项目需求描述

设计并实现一个期货账户分析网站，提供账户分析、交易统计、平仓记录、期末持仓和成交记录等功能模块。

2 整体方案

2.1 实现目标

（1）用户分析报告生成

提供自动化的账户分析报告生成功能，报告应包含详细的账户表现指标。

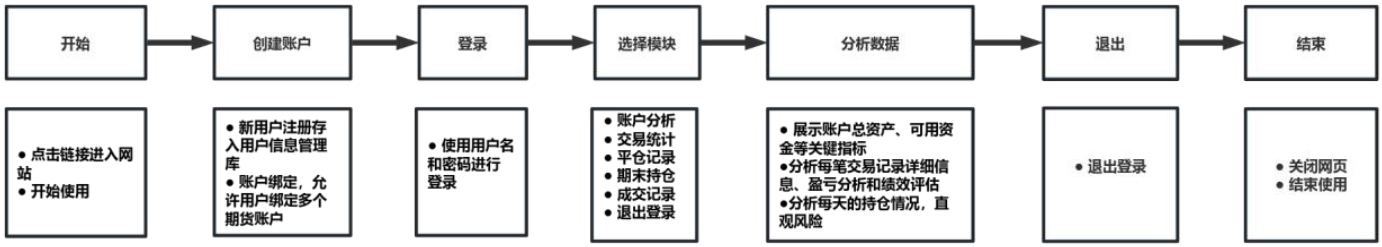
（2）数据统计与分析

通过对客户长期交易数据的统计与分析，为后续投资决策提供数据支持。

（3）用户界面与体验

设计直观、易用的用户界面，确保所有用户都能轻松导航和理解信息。并提供个性化设置，允许用户根据偏好调整报告格式和内容。

2.2 总体流程



2.3 产品架构与模块设计

期货账户分析网站旨在为期货交易者提供全面、实时的账户信息展示与深度数据分析服务。本设计采用分层架构模式，确保系统的可维护性、可扩展性和高性能。整体架构分为前端展示模块、后端逻辑处理模块、数据存储模块。

2.3.1 前端展示模块

（1）用户登录

用户名密码登录：新用户注册存入用户信息管理库

账户绑定：允许用户绑定多个期货账户

（2）主界面

可选下面五项，以及退出登录操作

（3）账户分析

账户总览：展示账户总资产、可用资金、持仓盈亏、保证金比例等关键指标。

交易日历：按时间轴展示每日的交易情况，包括开仓、平仓、手续费等。

资产分布图：以图表形式展示资产在不同品种、合约上的分布情况

（4）交易统计

汇总统计报告期内 多头、空头盈/亏前五名信息，以及每个合约在报告期内的交易手数、次数、以及多空交易情况，可以直观感受各合约盈亏

（5）平仓记录

交易记录：展示每笔交易的详细信息，包括成交时间、价格、数量、手续费等。

盈亏分析：计算并展示每日、每周、每月的盈亏情况，支持按品种、合约筛选。

绩效评估：基于夏普比率、最大回撤等指标评估交易绩效。

（6）期末持仓

统计每天的持仓情况，可以直观的了解账户持仓风险大小

（7）成交记录

汇总固定期限的交易记录情况

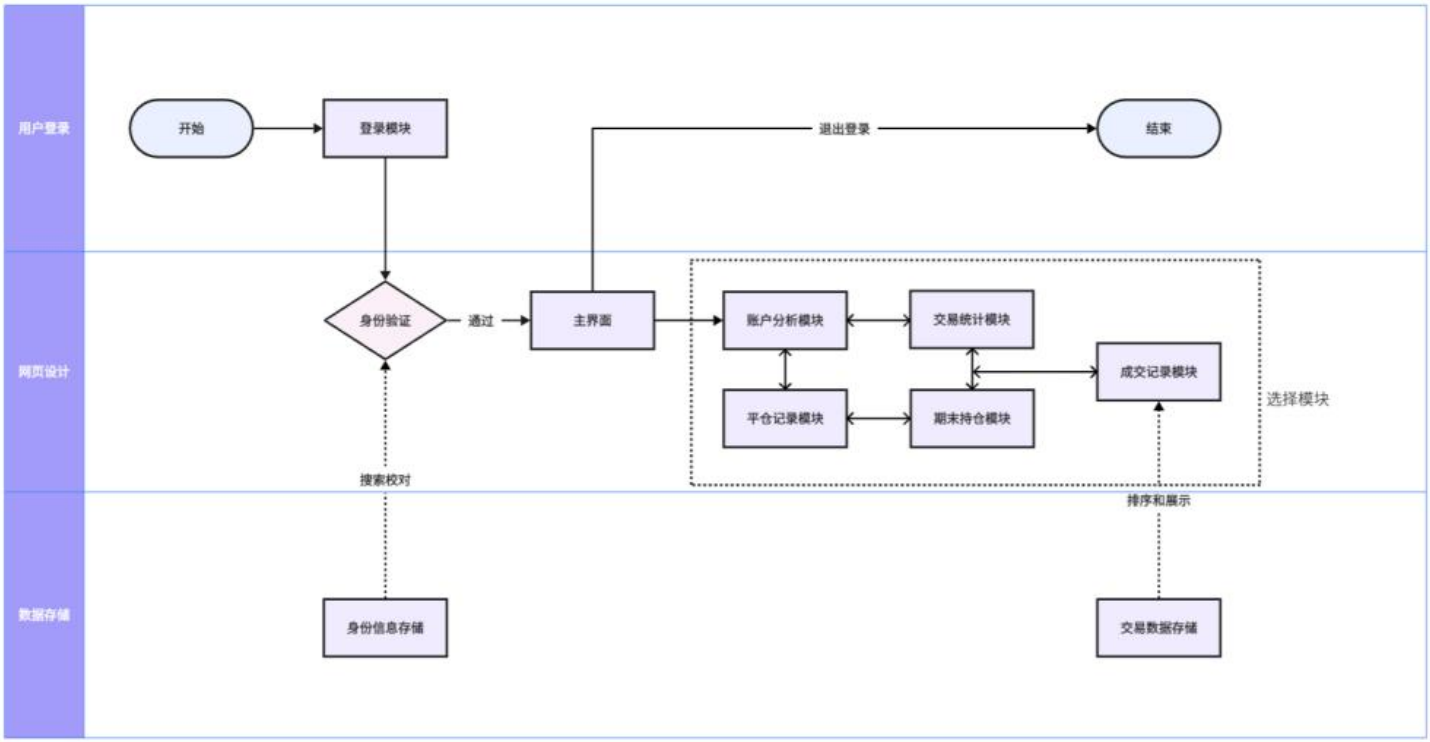
2.3.2 后端逻辑模块

一些工具类如后台基础数据的整合排序函数

API接口模块，如支持将账户数据、交易记录等导出为Excel等格式

2.3.3 数据存储模块

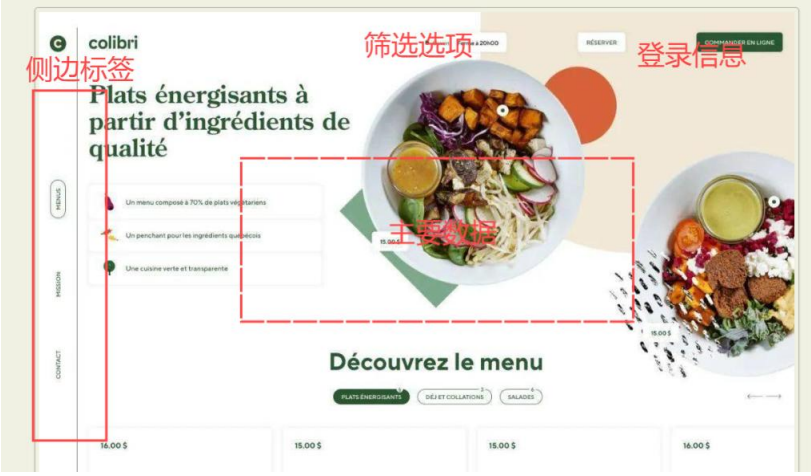
身份信息存储、交易数据存储



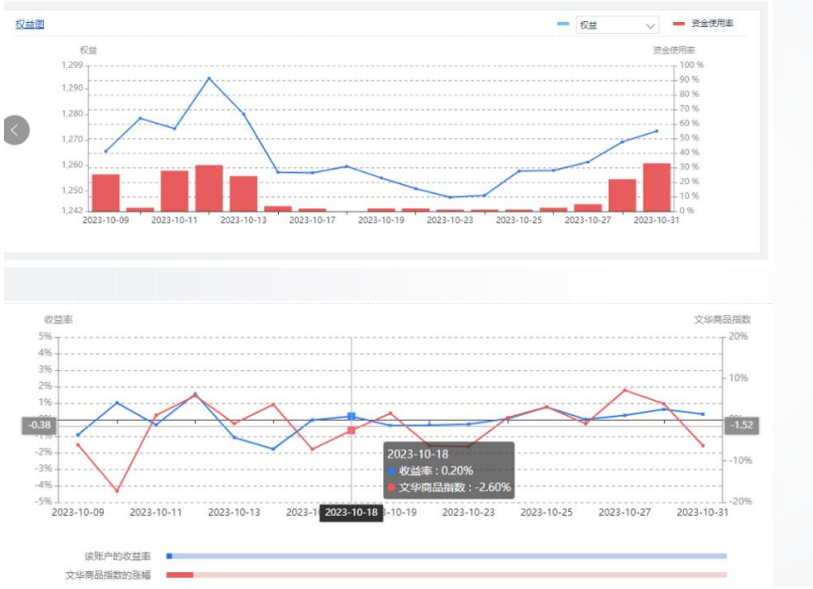
2.4 界面效果

登录界面排版可以参考下图，粗定简洁大标题

主界面展示所有模块的入口，默认停在账户分析界面，可采用侧边标签选项选择账户分析、交易统计、平仓记录、期末持仓、成交记录



每个模块界面展示相关数据的图表和统计信息。



3 开发设计

3.1 概要设计

3.1.1 组织结构

采用分层架构：采用经典的三层架构，包括表示层（前端）、业务逻辑层（后端）、数据访问层（数据库）。

3.1.2 功能

实现账户分析、交易统计、平仓记录、期末持仓和成交记录等功能。

3.1.3 处理流程

用户注册/登录流程

1. 用户提交注册信息。
2. 系统验证信息并创建用户档案。
3. 用户登录并进行身份验证。
4. 系统授权用户访问相应的服务。

交易流程

1. 用户提交交易订单。
2. 系统验证订单信息并执行交易。
3. 监控交易状态并更新用户持仓。
4. 生成交易报告并提供给用户。

资金管理流程

1. 用户发起资金调拨请求。
2. 系统验证资金状态并处理请求。
3. 更新用户资金流水和账户余额。
4. 提供资金管理报告。

3.1.4 模块间关系

前端通过API与后端交互，发送用户请求并接收处理结果。所有后端服务都与数据库模块交互，进行数据的增删改查操作。

3.2 详细设计

3.2.1 前端设计

1.页面排版布局设计：

（1）整体风格简洁大方，色彩搭配和谐，符合期货行业特点。

（2）模块划分清晰，方便用户快速找到所需信息。

- 顶部导航栏：包含网站logo、用户信息、报告生成按钮、设置选项。
- 侧边导航菜单：提供快速跳转至各功能模块的链接。
- 内容展示区：根据用户选择的模块，展示对应的数据和图表。
- 页脚：包含版权信息、联系方式、帮助链接。

（3）数据展示清晰，重点信息凸出：

- 使用表格、列表等元素展示数据，确保信息直观易懂。
- 对关键指标进行高亮处理，如收益率、盈亏等。

（4）数据可视化，图表效果展示：

- 使用图表库展示数据，提高信息传递效率。
- 图表类型包括：折线图、柱状图、饼图等，根据数据特点选择合适的图表。
- 实现图表的动态更新，以反映数据的实时变化。

2.数据获取和处理：

（1）数据来源：与后端接口对接，获取实时数据。

（2）数据绑定：将后端返回的数据绑定到前端页面的相应元素上，如输入框、表格、图表等。实现数据模型与视图的同步更新。

（3）数据处理：实现错误处理逻辑，包括网络错误、请求超时、API返回错误等。在用户界面上友好地展示错误信息，并提供相应的反馈或重试机制。

3.2.2 后端设计

（1）分层架构

后端采用典型的分层架构，通常包括以下几个层级：

- 表示层（路由层）：负责处理用户的请求，调用服务层的业务逻辑，将结果返回给前端。
- 服务层：处理核心业务逻辑，封装了与数据库交互的操作，提供API接口给表示层使用。

- 数据访问层：负责与数据库进行交互，执行增删改查操作。此层通过ORM框架对数据库进行操作。
- 工具层：封装一些常见的工具类和方法，如日期处理、数据格式化、日志记录等。

（2）数据库设计

- 用户表：存储用户的基本信息，如用户名、密码、权限等。
- 账户表：存储期货账户的基本信息，如账户ID、账户类型、余额等。
- 交易记录表：记录每一笔交易的详细信息，如交易时间、交易品种、买入/卖出方向、交易价格、数量等。
- 持仓表：记录账户当前的持仓情况，如品种、持仓数量、成本价等。
- 日志表：存储系统日志和用户操作日志，便于后期审计。

这些表之间可以通过外键、索引等关系进行关联，确保数据的一致性和查询效率。

（3）API设计

1. Excel文件生成：

   创建一个新的工作簿（Workbook）。

   创建一个工作表（Sheet）。

   填充数据到工作表中。

   将工作簿写入输出流。

2. 文件保存与下载：将生成的Excel文件保存到服务器的临时目录，并返回下载链接。或者直接通过HTTP响应发送文件内容。

3.3 开发周期

功能	子功能	负责人	预计开始	预计结束
前端展示	沟通网页，联调	申王鹤	20241128	20241202
	外观美化，信息排布	姜之楠	20241128	20241205
	图表展示	孟含桐	20241128	20241205
后端处理	逻辑接口	方童	20241128	20241203
	数据存储	李诗怡	20241128	20241202

4 非功能性需求

4.1 性能需求

（1）响应时间：系统应保证API的响应时间尽可能低，特别是对于实时数据查询（如账户信息、交易记录等），应保证在几百毫秒以内。

（2）数据一致性：尤其是在多用户访问、数据并发读写的情况下，要保证数据的一致性和正确性。对于关键交易数据的处理，尽量使用事务管理。

4.2 可用性与容错性

（1）高可用性：系统需要具备高可用性，确保在部分系统或硬件出现故障时，系统仍能继续运行。可以考虑部署多个实例，使用负载均衡和自动故障恢复机制。

（2）灾备机制：定期备份数据库，确保在发生灾难性故障时能够快速恢复系统。

（3）容错性：使用适当的错误处理机制，例如捕获异常并返回友好的错误消息。对于非关键操作（如日志记录），可以采用异步处理方式。

4.3 安全性

（1）身份认证与授权：所有用户操作都应该经过身份认证，敏感操作（如资金提现、账户修改）需要二次确认或管理员授权。

（2）数据加密：传输过程中的数据应通过HTTPS加密，以防止数据被窃取或篡改。同时，对存储的敏感数据（如密码）也应使用加密存储。

4.4 可维护性与可扩展性

（1）代码可维护性：遵循清晰的代码结构和命名规范，编写单元测试和集成测试，确保代码的可维护性。

如，变量名应用蛇形命名法（e.g. int flag_data）,函数名主要用驼峰命名，

而类名和文件名使用驼峰命名法（e.g. char* getNameString_p(),class NameBackend）

（2）可扩展性：系统设计时要考虑未来可能的扩展需求，使用微服务架构、插件机制等方式确保系统可以灵活扩展。

5 后续跟踪

5.1 用户体验优化

（1）界面设计：

响应式设计：确保网站在不同设备和屏幕尺寸上都能良好显示。

一致性设计：保持界面风格、色彩、字体等的一致性，提升用户体验。

交互优化：优化按钮、链接、表单等交互元素的反馈效果，提高用户操作的便捷性和准确性。

（2）功能增强：

个性化设置：允许用户自定义界面布局、数据展示方式等，满足个性化需求。

数据导出：提供数据导出功能，支持用户将账户分析报告导出为 PDF 、EXCEL等格式。

智能提醒：根据用户的交易行为和数据分析结果，提供智能提醒和建议。

5.2 数据保护

数据加密：对敏感数据进行加密存储和传输，如用户密码、交易数据等。

访问控制：实施严格的访问控制策略，确保只有授权用户才能访问敏感数据和功能。

5.3 安全防护

安全审计：定期进行安全审计和漏洞扫描，及时发现和修复安全漏洞。